University of Antwerp
Faculty of Arts

An Introduction to Command Line

DTA Bootcamp 2025 – 2026

Tutor: Jens Van Nooten

Goals for today's session

1. Learn what command line is
2. Learn our first commands
3. Write and run our first python script in command line

System Check

- **Did all Windows users install git bash (https://git-scm.com/downloads/win)?**

- **Did all Windows users activate WSL (https://winsides.com/how-to-enable-windows-subsystem-for-linux-in-windows/) and install Ubuntu (https://apps.microsoft.com/detail/9pdxgncfsczv)?**

Instructions for installing WSL and Ubuntu (WINDOWS!)

- In Powershell/terminal: `wsl --install`
- Go to https://ubuntu.com/desktop/wsl and follow the link to the Windows Store
- Open Ubuntu and wait for the installation to finish
- Enter (new) UNIX username and password
- All files created in Ubuntu will be stored under a separate directory
  - To access already existing files on your pc, type `cd /mnt/c`

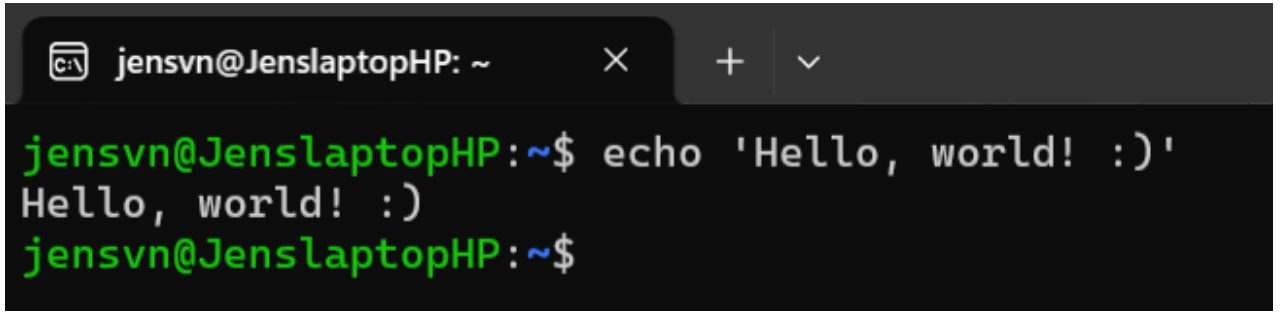
Command Line Interface, Terminals and Shells

What are they?

Command Line Interfaces

- **Command Line Interface (CLI):** Allows you to ‘talk’ and interact with your computer by entering commands
  - ⇔ Graphical User Interface (GUI)

- **Allows you to do a lot of things more efficiently:**
  - Navigating directories
  - Removing, renaming, creating files
  - Running programs/scripts
  - …



Terminals and Shells

- **Terminal:** tool that allows you to run commands
  - MAC terminal, Windows terminal…

- **Shell:** the interpreter of the commands you type
  - **Different shell, different commands**
  - MAC/Linux: bash
  - Windows: Powershell
    - That's why Windows users installed Ubuntu, which uses the bash shell!

Why use CLI?

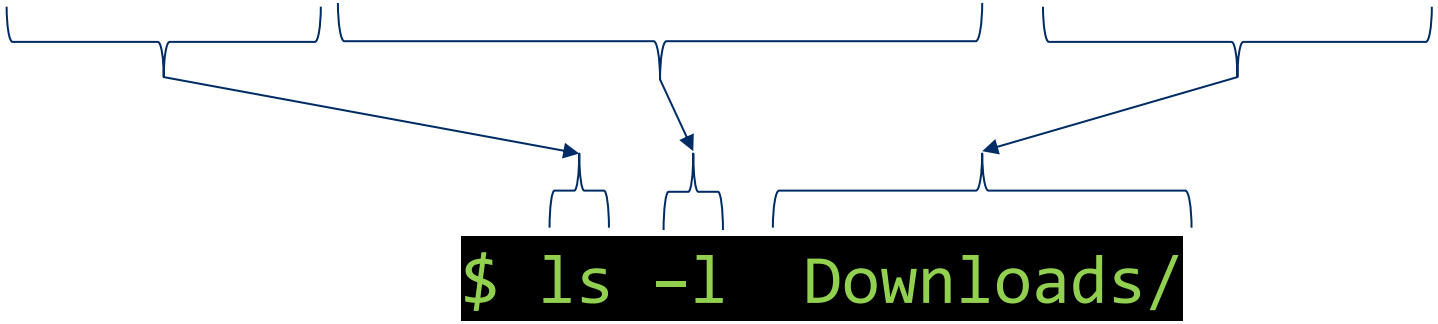
Why use it?

- **Efficiency:** Some tasks are faster to perform using commands rather than clicking through menus
  - Quickly rename incorrectly formatted filenames in a directory
- **Flexibility:** You can combine commands to perform complex tasks, automate processes, and access hidden system functionality
  - Schedule directory clean-ups, automatically zip files…
- **Power:** Many powerful tools for programming and data analysis work through the command line
- …

Let’s get into some commands!

Command structure

- **Basic structure of most commands you will use:**
  command [- flag/option(s) [value]] [arguments]



- $ = beginning of command, don't type this ☺
- **A command can have zero or more options**
  - An option can have zero or one value
- **A command can have zero or more arguments**
- **Arguments always come after options**
- **Redirection is optional**

Some basic commands: pwd

- **pwd: shows the current working directory (where you are in your system)**
  - First / : root, or the top most directory in your system

```
jensvn@JenslaptopHP:~$ pwd
/home/jensvn
```

- **Keep in mind: commands are case sensitive! Pwd =/= pwd =/= PWD**

Some basic commands: ls

- **`ls`*: Shows a list of files and directories***
  - Multiple 'flags' (command options):
    - `-l`: shows permissions, number of links/directories in this dir, user, group, size in bytes, date of last modification and name
    - `-a` to include hidden files
    - `-s` to include file size
    - `-R` to recursively list directory tree

```
jensvn@JenslaptopHP:~$ ls -l
total 12
-rw-r--r-- 1 jensvn jensvn   40 Feb 13  2024 test1.txt
-rw-r--r-- 1 jensvn jensvn   21 Feb 13  2024 test2.txt
drwxr-xr-x 2 jensvn jensvn 4096 Sep 29 15:02 test_dir
```

- **Also works with**
  - specific directories: `ls test_dir/`
  - wildcards: `ls *.txt`

Some basic commans: cd

- **Let's get moving! Use `cd` to go down or up in your filesystem**
  - First use `ls` to see which directories you can move to
  - Then, do `cd [DIRECTORY]`
    - cd `test_directory`
  - Notice how the path changed from ~ to ~/test_dir !

```
jensvn@JenslaptopHP:~$ ls
test1.txt  test2.txt  test_dir
jensvn@JenslaptopHP:~$ cd test_dir/
jensvn@JenslaptopHP:~/test_dir$
```

Some symbols to know

- **Relative paths:**
  - `.` : current location or ‘any character’ when searching for string patterns (= regex)
  - `..` : parent directory
  - `~` : home directory in a path (user-specific; not the root!!!)
    - ~/path/to/directory

- `*` **: ‘all’; wildcard can be combined with strings, including extensions**
  - *.py = all files ending in py
  - *test* = all files that contain ‘test’
  - …

Some symbols to know

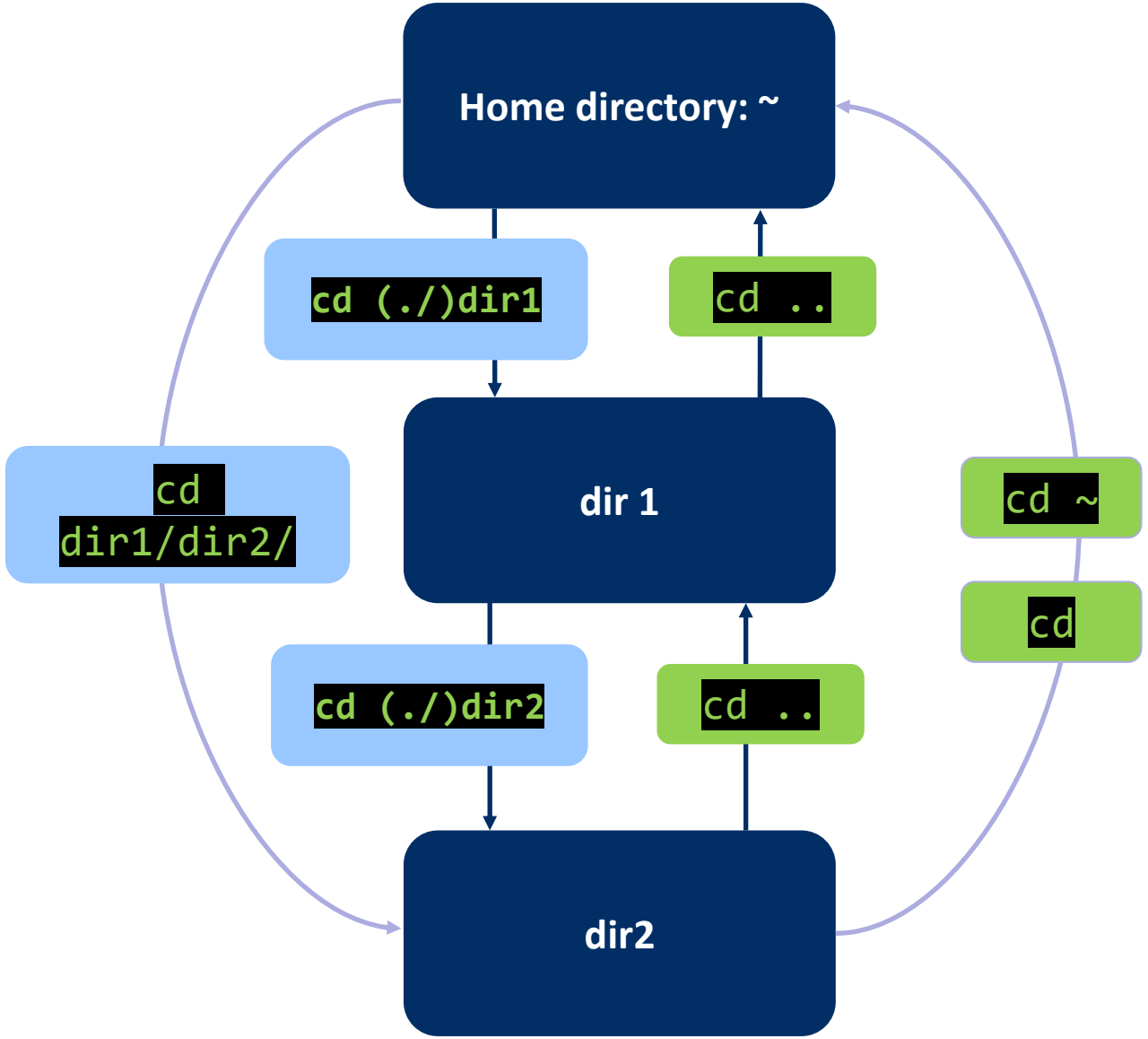
- So, **moving back up**: `cd ..`
- Moving to your home directory: `cd ~` or `cd`
- You don't always need to specify the full path from the root up:

```
cd /home/User/dir1/dir2
```

=

```
cd ./dir1/dir2
```

  - You can even leave out `./` in this case



Some Basic Commands: mkdir, touch and nano

- **Let's make some directories and files!**
  - `mkdir` : allows you to create an empty directory (in working directory)
    - `mkdir test_dir`
  - `touch`: creates an empty file (in working directory)
    - `touch test_file.txt`
    - `touch test_dir/test_file.txt`
  - `nano`: creates an empty file if file doesn't exist already
    - Otherwise it opens the existing file
    - `nano test_file.txt`

Excursion: Redirecting and Appending

- You can also create files by redirecting commands (>):
  - `ls test_dir > output_ls.txt`
  - `echo ‘hello, world!’ > hello.txt`

- Appending is quite easy (>>):
  - `echo ‘goodbye, world!’ >> hello.txt`

Some Basic Commands: nano, cat, head, tail

- **Now that we've created some files, we want to view them**
  - `nano`: opens file in editor
  - `cat`: print out content
  - `head`: show n first lines of file
    - head test_file.txt
    - head –n 5 test_file.txt
  - `tail`: show n last lines of file
    - tail test_file.txt
    - tail –n 5 test_file.txt

```
  GNU nano 6.2
Hello, this is a test! I'm a file :)
```

```
jensvn@JenslaptopHP:~$ cat test_file.txt
Hello, this is a test! I'm a file :)
```

Some Basic commands: cp and mv

- **Backing up your files is always a good idea:**
  - Copying file(s)/dir(s): `cp`
  - Comes with multiple options, including `-r` (recursive)
  - `cp [FILE_NAME] [COPIED_FILE_NAME]`
    - `cp test_file.txt test_file_copy.txt`
    - `cp test_file.txt test_dir/test_file_copy.txt`
    - `cp -r test_dir test_dir_copy`

- **Moving files: `mv`**
  - Similar syntax to cp
  - Recursive: `-r`
  - `mv [FILE] [DESTINATION]`
    - `mv test_file.txt test_dir/test_file.txt`

Some Basic Commands: rm

- **BE VERY CAREFUL WITH THE FOLLOWING COMMANDS! REMOVING FILES CANNOT BE UNDONE!**
  - Always doublecheck **where** you are in your system
  - Doublecheck **what** you want to remove
  - ***Make back-ups or use trash (or other variants)***

- **Removing files can be done with `rm`**
  - `rm test_file.txt`
  - Recursive: -r
    - Removes directory and all files in it: `rm –r test_dir`

- **Removing an empty directory: `rmdir`**
  - `rmdir test_dir`

Running a python script

- **Let's create a python script!**
  - Open an empty file (`touch`, `nano`…)
  - Write a very basic script (print something)
  - Save and close the file (ctrl/cmd + x, then press 'y')
  - Run the file (`python test_script.py`)

In summary

- **Moving around:** `cd`
- **Creating files and dirs:** `mkdir, touch, nano`
- **Opening files:** `nano, vi(m)`
- **Copying files and dirs:** `(s)cp`
- **Moving files and dirs:** `mv`
- **Running a python script:** `python / python3`

Let’s go deeper!

Other use cases for CLI

- **Version control of code:** `git`
- **Downloading packages:** `brew / winget`
- **Downloading python packages:** `pip`
- **Anaconda-related commands:** `conda`
- **Transfer data:** `curl`
- **Testing your network connection:** `ping`
- **Connecting to a remote server:** `ssh`

Other useful commands

- **Look up these commands:**
  - `grep` : lookup in files
  - `wc` : get word counts
  - `less` : open file in new window

- **Sequentially executing commands: pipelines**
  - Multiple commands separated by `|`
    - `command_1 | command_2`
    - `ls |grep ‘test’`

Bonus: Path

- **How does your terminal know which commands it can run?**
  - Path! (usually located in `/usr/bin/` or `/usr/local/bin/`)
  - Directory that keeps track of all the commands you can run

- **If you try to run a command in a terminal:**
  - It checks the path
  - If the corresponding directory for the command is found, the command is executed

Bonus: Path

```
jensvn@JenslaptopHP:~$ cd /usr/bin/
jensvn@JenslaptopHP:/usr/bin$ ls
 NF                     cpp-11                               gcov-11                 lsattr        piconv                sha224sum         uname
'['                     crontab                              gcov-dump               lsb_release   pidof                 sha256sum         unattended-upgrade
 aa-enabled             csplit                               gcov-dump-11            lsblk         pidwait               sha384sum         unattended-upgrades
 aa-exec                ctail                                gcov-tool               lscpu         pinentry              sha512sum         uncompress
 aa-features-abi        ctstat                               gcov-tool-11            lshw          pinentry-curses       shasum            unexpand
 add-apt-repository     curl                                 gdbus                   lsipc         ping                  showconsolefont   unicode_start
 addpart                cut                                  gencat                  lslocks       ping4                 showkey           unicode_stop
 addr2line              cvtsudoers                           geqn                    lslogins      ping6                 shred             uniq
 apport-bug             dash                                 getconf                 lsmem         pinky                 shuf              unlink
 apport-cli             date                                 getent                  lsmod         pip                   size              unlzma
 apport-collect         dbus-cleanup-sockets                 getkeycodes             lsns          pip3                  skill             unshare
 apport-unpack          dbus-daemon                          getopt                  lsof          pip3.10               slabtop           unsquashfs
 apropos                dbus-launch                          gettext                 lspci         pkaction              sleep             unxz
 apt                    dbus-monitor                         gettext.sh              lspgpot       pkcheck               slogin            update-alternatives
 apt-add-repository     dbus-run-session                     ginstall-info           lsusb         pkcon                 snap              update-mime-database
 apt-cache              dbus-send                            gio                     lto-dump-11   pkexec                snapctl           uptime
 apt-cdrom              dbus-update-activation-environment   gio-querymodules        lzcat         pkill                 snapfuse          usb-devices
 apt-config             dbus-uuidgen                         git                     lzcmp         pkmon                 snice             usbhid-dump
 apt-extracttemplates   dd                                   git-receive-pack        lzdiff        pkttyagent            soelim            usbreset
 apt-ftparchive         deallocvt                            git-shell               lzegrep       pl2pm                 sort              users
 apt-get                deb-systemd-helper                   git-upload-archive      lzfgrep       pldd                  sotruss           utmpdump
 apt-key                deb-systemd-invoke                   git-upload-pack         lzgrep        plymouth              splain            uuidgen
 apt-mark               debconf                              glib-compile-schemas    lzless        pmap                  split             uuidparse
 apt-sortpkgs           debconf-apt-progress                 gmake                   lzma          pod2html              splitfont         vdir
```

Lifelines

- ✓ You can write `man` before most default command names to get a comprehensive manual for the command
- ✓ Alternatively, you can use the option -h to get a quick guide to the correct usage of that command
- ✓ Pressing Ctrl/CMD+C terminates whatever process you're in
- ✓ You can cycle through your previous commands by pressing the up and down arrows
- ✓ You can use the tab key to autocomplete commands and filenames
- ✓ **Be VERY careful when asking ChatGPT for help!!!**

Summary and next steps

- **Check these resources for extra info and exercises:**
  - **Command cheat sheet**: https://devhints.io/bash
  - **Terminal tutorial**: https://ubuntu.com/tutorials/command-line-for-beginners#3-opening-a-terminal
  - **Ubuntu for everyone**: https://cocalc.com/projects/a3734386-82ac-45f1-81b5-dff7f696cecb/files/welcome%20to%20CoCalc.term

- **Practice with simple exercises!**
  - Create directories, move them…
  - Create simple python scripts, run them, redirect the output…

- Note: **be careful when copying code from forums or ChatGPT!** If not, you might seriously mess up your system. Always double check code before executing!
- **BE VERY CAREFUL WITH rm!!!**


jensvn@JenslaptopHP:~$ cowsay Now get practicing!
< Now get practicing! >
\ ^__^
\ (oo)_______
(__)\ )\/\
||----w |
|| ||